

COMP2200/COMP6200 Practical Exercise (Week 11): Network Analysis

School of Computing

Preparation

Choose your environment: either run locally with `uv` or use Google Colab.

1. Local with `uv`:

- (a) Install `uv` if you do not already have it: `uv` installation instructions.
- (b) Create a folder for this practical (for example, `week11-prac`) and open a terminal in it.
- (c) Set up your environment:

```
uv init
uv add networkx matplotlib pandas numpy scipy openpyxl
```

- (d) Launch Jupyter with `uv run --with jupyter jupyter lab` (or ... `notebook`).

2. Google Colab:

- (a) Open Colab and create a new notebook.
- (b) Upload `TopRoutesJuly2014Jul2025.xlsx` from iLearn or this week's zip file. If you are using the sample class data instead of the live spreadsheet, also upload `class_network.csv`.
- (c) Install packages with `!pip install networkx matplotlib pandas numpy scipy openpyxl`.

Part A — Class network exercise (40 min)

In this practical, we will form a small network using the connections between students in this prac, and then we'll look at a much bigger network (passenger routes between Australian airports).

A1. Spreadsheet and board network warm-up

Before we do this in Python, fill in the shared class network spreadsheet on SharePoint. The prac tutor or demonstrator will then use the completed rows to draw a small class network on the board. With at most 15 students in a prac, this gives everyone one shared example to refer to.

1. Open the default SharePoint spreadsheet, unless your prac leader has another URL for you:
COMP2200 Week 11 class network spreadsheet.
2. Look for the worksheet tab with your prac group. Add a row with a pseudonym and the unit codes you are studying this semester, leaving out COMP2200 and COMP6200.
3. The tutor reads the completed rows and writes student pseudonyms down one side of the board and unit codes down the other side.
4. Draw an edge from each student to each unit they study. This gives a bipartite graph: student nodes on one side, unit nodes on the other.
5. Pick two students and find a shortest path between them by hand. Start at the first student, then move along board edges one step at a time until you reach the target student.

6. In a second colour, sketch the projected student graph: connect two students directly if they share at least one unit, and write the number of shared units on the edge if there is space.
7. Which unit or student looks most central? What happens if you remove that node and its edges? Does the network split into separate pieces?

This is the same idea as the code below: NetworkX will store the spreadsheet labels as nodes and the board lines as edges.

A2. Import required libraries

First import the Python modules we need: we'll need `math`, `numpy`, `networkx`, `pandas` and `matplotlib`.

```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import math
```

A3. Load the class network data

Use the same SharePoint spreadsheet that the class just filled in. If your prac leader has another URL for you, use that instead.

When everyone has finished entering data, download a copy of the spreadsheet from SharePoint/Excel. The exact menu wording varies, but look for the option to download a copy as an Excel workbook. Keep the worksheet tab for your prac group.

Using `pandas`, load the spreadsheet and build a bipartite graph where students and units are both nodes, with an edge from each student to each unit they study other than COMP2200/COMP6200.

```
# Load the spreadsheet (adjust filename and sheet name as needed).
# The sheet name should be your prac group tab.
sheet_name = 'Wed 11am prac'
class_df = pd.read_excel('class_network.xlsx', sheet_name=sheet_name)

# If you are using the bundled sample class data instead:
# class_df = pd.read_csv('class_network.csv')
```

If you get an error about `openpyxl` not being available, fix that with whatever tool you use to manage packages (`pip install openpyxl` or `uv add openpyxl` or `conda install openpyxl`).

A4. Build the class network graph

These are the columns that contain each student's units:

```
unit_cols = ['Unit1', 'Unit2', 'Unit3', 'Unit4']
```

Create an empty graph:

```
G_classes = nx.Graph()
G_classes.add_nodes_from(class_df['Name'])
```

Add an edge between a student and each unit they study. You can do this by iterating over every element in `unit_cols` and creating an edge between the 'Name' and that unit column.

```
for unit in unit_cols:
    for name, u in zip(class_df['Name'], class_df[unit]):
        if pd.notna(u):
            G_classes.add_edge(name, u)
```

A5. Project to a student-only network

The bipartite graph keeps the modelling choice honest: student–unit–student paths show which unit created the connection. For some questions, though, it is useful to project the graph onto students only. In the projected graph, two students are connected directly when they share at least one unit, and the edge weight counts how many units they share.

```
from networkx.algorithms import bipartite

student_nodes = set(class_df['Name'])
student_graph = bipartite.weighted_projected_graph(G_classes, student_nodes)

student_graph.number_of_nodes(), student_graph.number_of_edges()
```

A6. Find paths in the network

Use `nx.shortest_path` to find the path between yourself and another student in the bipartite graph. Then check whether those two students have a direct shared-unit edge in the projected graph.

```
path = nx.shortest_path(G_classes, 'YourName', 'TheirName')
print('Shortest path:', path)

if student_graph.has_edge('YourName', 'TheirName'):
    print('Shared units:', student_graph['YourName']['TheirName']['weight'])
```

A7. Visualise the class network

Create layouts for the bipartite class network and the projected student network, and draw them using Matplotlib.

```
pos_class = nx.spring_layout(G_classes)
plt.figure(figsize=(8,8))
nx.draw_networkx(G_classes, pos=pos_class, node_size=20, alpha=0.7)
plt.title('Class network: students and units')
plt.show()

pos_students = nx.spring_layout(student_graph)
plt.figure(figsize=(8,8))
nx.draw_networkx(student_graph, pos=pos_students, node_size=80, alpha=0.7)
plt.title('Projected student network')
plt.show()
```

A8. Characterise the network

Calculate the average clustering coefficient and the average shortest path length in both graphs. The bipartite graph should have clustering 0 because it has no triangles. The projected student graph is the graph where clustering among students can appear.

```
def avg_path_or_largest_component(graph):
    if nx.is_connected(graph):
        return nx.average_shortest_path_length(graph)
    subgraph = graph.subgraph(max(nx.connected_components(graph), key=len))
    return nx.average_shortest_path_length(subgraph)

avg_clust_bipartite = nx.average_clustering(G_classes)
avg_shortest_bipartite = avg_path_or_largest_component(G_classes)

avg_clust_students = nx.average_clustering(student_graph)
avg_shortest_students = avg_path_or_largest_component(student_graph)
```

```

print('Bipartite clustering:', avg_clust_bipartite)
print('Bipartite average path:', avg_shortest_bipartite)
print('Student projection clustering:', avg_clust_students)
print('Student projection average path:', avg_shortest_students)

print('N: ', G_classes.number_of_nodes())
print('log(N): ', math.log(G_classes.number_of_nodes()))

```

Based on the projected student graph, does the class network look more like:

- a **random graph** (low clustering, average path around $\log(N)$)
- a **small world** network (high clustering, average path around $\log(N)$)
- a **chain** (average path grows with N , low clustering)
- or **other**?

Part B — Airport network analysis (50 min)

Downloaded from the BITRE domestic airline activity time series.

B1. Load the airport data

Read the Excel file (TopRoutesJuly2014Jul2025.xlsx) that lists the top airline routes. You'll need to specify the sheet name as "Top Routes" and skip the first 12 rows.

```

raw_route_information = pd.read_excel('TopRoutesJuly2014Jul2025.xlsx',
                                     sheet_name='Top Routes',
                                     skiprows=12)

raw_route_information

```

B2. Clean the flight data

There is a column called "City Pair Route.1" which has the origin and destination airports.

- Drop the nulls
- Occasionally it has a newline character '\n' instead of a space, so replace those
- Drop the duplicates

```

flights = (raw_route_information['City Pair Route.1']
          .str.replace('\n', ' ')
          .drop_duplicates()
          .dropna())

flights

```

B3. Build the airport network

Create a graph for the airport network.

```
airport_network = nx.Graph()
```

Add an edge for each flight. You will need to iterate over the values in the flights column and split the string on ' - '

```

for flight in list(flights):
    source, dest = flight.split(' - ')
    airport_network.add_edge(source, dest)

```

Check how many airports and routes are in the network:

```
airport_network.number_of_nodes(), airport_network.number_of_edges()
```

B4. Visualise the airport network

Create a layout for the network visualization.

```
pos = nx.spring_layout(airport_network)
```

Draw the nodes and edges using Matplotlib.

```
plt.figure(figsize=(12,12))
nx.draw_networkx(airport_network, pos=pos, node_size=10,
                 alpha=0.5, with_labels=True)
plt.title('Australian Airport Network')
plt.show()
```

B5. Average clustering and path length

Calculate the average clustering coefficient.

```
avg_clust = nx.average_clustering(airport_network)
print('Average clustering:', avg_clust)
```

Find the average shortest path length. For disconnected graphs, use the largest connected component.

```
if nx.is_connected(airport_network):
    avg_shortest = nx.average_shortest_path_length(airport_network)
else:
    subgraph = airport_network.subgraph(
        max(nx.connected_components(airport_network), key=len)
    )
    avg_shortest = nx.average_shortest_path_length(subgraph)
print('Average shortest path length:', avg_shortest)
```

Calculate N and $\log(N)$:

```
print(airport_network.number_of_nodes())
print(math.log(airport_network.number_of_nodes()))
```

Based on the last two outputs, does this airport network look more like:

- a **random graph** (low clustering, average path around $\log(N)$)
- a **small world** network (high clustering, average path around $\log(N)$)
- a **chain** (average path grows with N , low clustering)
- or **other**?

B6. Is it a scale-free (ultra small world) distribution?

Calculate the degree of each airport and plot the distribution on a log scale.

```
degrees = [d for _, d in airport_network.degree()]
degrees_sorted = sorted(degrees, reverse=True)
plt.figure()
plt.loglog(np.arange(len(degrees_sorted)) + 1, degrees_sorted,
           marker='o', linestyle='None')
plt.xlabel('Rank')
plt.ylabel('Degree')
plt.title('Degree Distribution (log-log)')
plt.show()
```

Does the log-log graph look linear? If so, this suggests a scale-free network with a power-law degree distribution.

Part C — Centrality measures (30 min)

C1. Degree centrality

Compute degree centrality. It returns a dictionary; it is often convenient to turn it into a Series and only show the top few.

```
pd.Series(nx.degree_centrality(airport_network)).nlargest(5)
```

C2. Betweenness centrality

Compute betweenness centrality.

```
pd.Series(nx.betweenness_centrality(airport_network)).nlargest(5)
```

C3. Closeness centrality

Compute closeness centrality.

```
pd.Series(nx.closeness_centrality(airport_network)).nlargest(5)
```

C4. PageRank

Compute PageRank.

```
pd.Series(nx.pagerank(airport_network)).nlargest(5)
```

C5. Eigenvector centrality

Compute eigenvector centrality.

```
pd.Series(nx.eigenvector_centrality(airport_network)).nlargest(5)
```

C6. Discussion questions

1. Based on the centrality measures above, which airport is the most **central** or influential in this network?
2. There are differences in the **second** most central node. What is that reflecting?

Part D — Ego networks and communities (20 min)

D1. Ego network of the center of the network

We can get a bit of an understanding of the core of the network by picking a very central node and then creating an ego network from it.

```
ego_net = nx.ego_graph(airport_network, 'Sydney')
plt.figure(figsize=(6,6))
pos = nx.spring_layout(ego_net)
nx.draw_networkx(ego_net, pos=pos, node_size=20,
                  alpha=0.7, with_labels=True)
plt.title(f'Ego network for Sydney')
plt.show()
```

D2. Try an alternate visualisation

Draw the same ego network using a shell layout.

```
nx.draw_shell(ego_net, node_size=10)
```

D3. Community detection

Sometimes we don't want to focus on the core of the network – sometimes we want to get an overview of the whole network.

Run greedy modularity to find communities.

```
from networkx.algorithms import community
communities = community.greedy_modularity_communities(airport_network)
```

Print the number of communities and their members.

```
print('Number of communities:', len(communities))
for c in communities:
    print(c)
```

Do the detected communities make sense geographically? Do they cluster by region (e.g., Queensland routes, NSW routes, WA routes)?

Discussion Questions

1. How do the class network and airport network differ in their structure?
2. Which centrality measure do you think best captures “importance” for airports? Why?
3. What does the community structure tell you about Australian domestic aviation?
4. If you were to remove the most central airport from the network, how would that affect connectivity?

Quiz questions

As usual, carry on with some quiz questions.